

Assignment 1: Deep Learning

1st Kian Niek Hamidi Abd Abad
Utrecht University
Utrecht, Netherlands
0609927

2nd Márton Mohácsi
Utrecht University
Utrecht, Netherlands
1621106

3rd Sid Pranjale
Utrecht University
Utrecht, Netherlands
4368959

4th Jorik Roebroek
Utrecht University
Utrecht, Netherlands
9490466

Abstract—This document outlines the methodology and model selection process for predicting non-stationary laser measurement time-series data. We compare Multi-Layer Perceptron (MLP), Convolutional Neural Network (CNN), and hybrid CNN-LSTM architectures. Our findings demonstrate that a window size of 520 steps is necessary to capture periodic signal collapses. The final hybrid model achieved an MSE of 815.17 and an MAE of 20.77 on the 200-step recursive test set.

Index Terms—Deep Learning, Neural Networks, Time Series Forecasting

I. INTRODUCTION

The problem we were asked to solve was predicting laser measurements on a real-life dataset, visualized in Figure 2. This involved predicting the next measurement based on the sequence of the previous N measurements. The sequence of N values that are used as an input will be referred to as a window from now on. This can be seen illustrated in Figure 1.

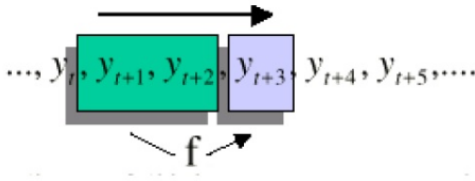


Fig. 1. Illustration of the inputs and outputs of the windowed prediction

To solve this problem we had to find one or more neural network architectures to use. Then different window sizes had to be tried to find the best solution.

The models were tested on predicting the next 200 data points recursively, by feeding the outputs of it into the input of the next step.

Through comparative testing, we found that a hybrid CNN-LSTM architecture utilizing a window size optimized through a parametric sweep was the only model capable of maintaining the signal's complex growth-and-collapse patterns during long-term recursive forecasting.

II. USED MODELS

We developed a MLP, CNN, and RNN(LSTM) model in parallel to see which showed the most promise for solving this task.

A. MLP

The MLP (Multi-layer perceptrons) model is conceptually the simplest model, but can achieve relatively good results. We chose it because it is simple to implement and requires relatively few design choices and hyper-parameters. Its main downside is that the number of parameters quickly increases for larger input and layer sizes, which means that the window size has to be quite small for this model. This will likely result in the model not being able to fully capture the largest recurring pattern in the data.

B. CNN

We also tried an approach with a simple CNN (Convolutional Neural Network) model. We chose this model because it solves the problem of the MLP with the small window size, as a convolution kernel with a smaller number of parameters is used instead of some of the fully connected layers, allowing for larger window sizes. While CNNs do not have a special quality with regard to time-series data, we hoped that the model would be able to extract the different patterns of the data, like the oscillation, growing amplitude, and regular collapse.

C. LSTM

Finally, we made an LSTM (Long Short-Term Memory) model, which we initially chose because of its ability to work well with time-series related data. Initially the results of this network were great when predicting in the same time frame of the training data. However having the model predict the data after the time frame of the training data; the amplitude of the predictions quickly stagnated, failing to capture the macro-level evolution of the signal amplitude shown in Figure 2.

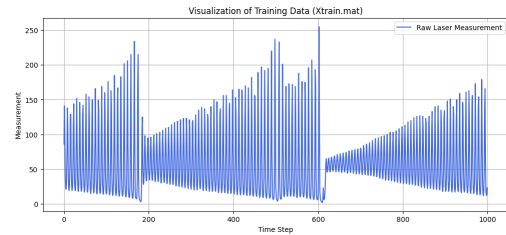


Fig. 2. Training Data Visualized in a graph with time step on the X axis and measurement value on the Y axis

To make the model be better at adapting to the macro shape of the data we added a singular convolutional layer to help the model capture the macro shape.

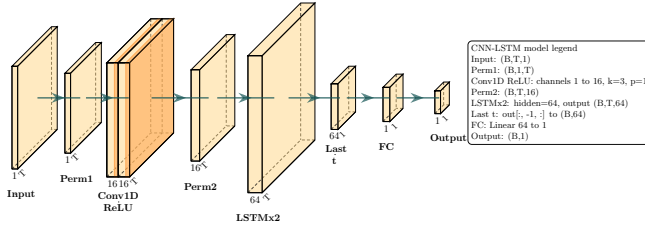


Fig. 3. Visualization of the LSTM network with the convolution layer

III. APPROACH

A. Data

The measurements contained 1000 instances of sequential data, between the values of 0 and 260. This can be seen in Figure 2. Since we only had a limited number of measurements, and we did not know how big these values might get, we normalized the data using a Min-Max scaler to a range of $[-1, 1]$ to facilitate gradient convergence.

Then datasets of different window sizes were created. This was done by taking a data point and the N values preceding it and pairing them up. These were saved, and loaded depending on the window size that was to be tried out. In the case of a window size of N and a data point at time t , where $t < N$ the data point was not used for either validation or training.

The data contains three major growing phases and two collapses. Having a validation set that is the sequence of paired data could lead to only having samples in one of the growing phases and completely missing the collapses. This would prevent the model from converging correctly since the validation set would not have data that is general for the dataset. Because of this the training and validation sets were created by randomly splitting the paired data.

B. MLP

For the MLP model, we trained for a maximum of 300 epochs with a batch size of 32 and learning rate of 0.001, using MSE loss and the Adam optimizer. We chose MSE over MAE, as it penalizes larger differences more, which will put more pressure on the model to learn the amplitude collapses in the data. We also employed an early stopping mechanism on the loss of the validation set, with a patience of 8. The model itself consists of 4 fully-connected layers, including input and output:

- Input: size 32 (The final window size)
- Hidden layer 1: size 64
- Hidden layer 2: size 32
- Output: size 1

Since the available training data is very limited, we wanted to keep the model very small, thus opting for only two hidden layers. We tried several different sizes for the hidden layers, finding that the current configuration produced the best results. Likewise, we tested window sizes from 16 to 128 in powers of 2, finding that 32 performed best. Larger window sizes were

shortly looked at, but resulted in the model failing to properly converge during training.

C. CNN

For the CNN model, we used the same training method as the MLP. The model itself is structured as follows:

- Convolution layer 1: input 64×1 , #filters 16, kernel 15
- Convolution layer 2: input 50×16 , #filters 16, kernel 15
- FC output layer: size 1

Initially our CNN model only contained a single convolution layer, but the training loss was quite high, and the best value for the kernel size ended up being the size of the input, which means it essentially functioned as a large MLP at that point. Adding a second convolution layer greatly improved the results with small kernel sizes. For simplicity, we kept the same kernel size and number of filters for both convolutional layers. While for this model was to be able to handle bigger window sizes than the MLP, the model still failed to converge when the window size was above 100. The optimal window size we found was 64.

D. The CNN-LSTM Approach

As mentioned in II-C the proposed architecture utilizes a CNN-LSTM model designed to process sequential data. The overall training configuration is set to run for 100 epochs with a batch size of 32 and a learning rate of 0.001, utilizing a CUDA-enabled GPU for accelerated computation.

The model's data pipeline and layer architecture are structured as follows:

- **Input Representation:** The network ingests the initial data as a tensor of shape $(B, T, 1)$ where B is the batch which is the number of separate sequences processed at once, T is the time which is the number of observations per sequence and, the 1 stands for the C which is the Feature which is the number of variables measured at each observation. Based on the implementation, the sequence length (T) is defined as 520, and the batch size (B) is set to 32.
- **Convolutional Feature Extraction:** To capture local patterns, the input tensor is first permuted to the shape $(B, 1, T)$. It is then passed through a 1D Convolutional layer (Conv1D) that utilizes a kernel size of $k = 3$ and a padding of $p = 1$. This convolution maps the single input channel to 16 output channels and is immediately followed by a ReLU activation function.
- **LSTM Processing:** Following feature extraction, the tensor is permuted back to $(B, T, 16)$ to serve as a valid sequence for the recurrent layers. The data is processed by a Long Short-Term Memory (LSTM) network consisting of 2 layers and a hidden size of 64. This stage yields an intermediate output tensor of shape $(B, T, 64)$.
- **Prediction and Output:** To formulate the final prediction, the model extracts the LSTM output from the final time step (`out[:, -1, :]`), effectively reducing the tensor dimensions to $(B, 64)$. Finally, a Fully Connected

(Linear) layer maps the 64 hidden units to a single output, resulting in the final prediction shape of $(B, 1)$.

This was roughly inspired by the work of Abumohse M. et al. [1]

A critical component of the methodology involved tuning the window size (N). We conducted a parametric sweep of N values ranging from 200 to the total length of the dataset in increments of 25. This was necessary because a window that is too short fails to capture the "collapse" cycles, while a window that is too large reduces the number of available training samples. The final recursive evaluation demonstrates that a balanced sequence length allows the model to maintain the signal's structural integrity over a 1,000-step horizon. Based on this sweep, an optimal window size of $N = 520$ was selected for the final model.

IV. RESULTS

The models' performance was evaluated using two primary metrics: Mean Squared Error (MSE) and Mean Absolute Error (MAE).

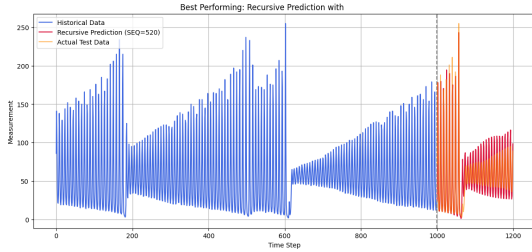


Fig. 4. Recursive Prediction integrated with training Data (in blue). The vertical dashed line at $t = 1000$ marks the transition from training (historical) observations to the recursive forecast, demonstrating the model's ability to maintain oscillating patterns over a longer time period.

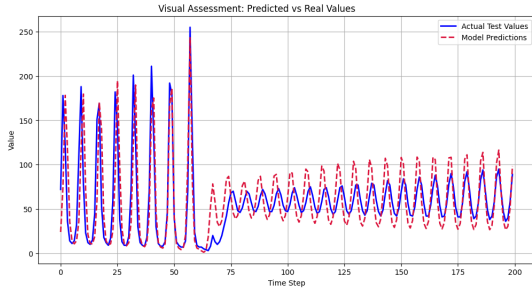


Fig. 5. Visual comparison of Model Predictions against Actual Test Values over 200 time steps. The model closely tracks the periodic peaks, though it exhibits slight overestimation in the latter half of the test sequence.

A. Choice of model

Comparing the results of the different models, we found that the MLP and CNN failed to properly capture the period of the large collapse pattern in the data. Both models tend to collapse by a smaller degree and therefore repeated the collapse too frequently. The CNN-LSTM model was able to predict this much better. Therefore, we continued with this model for the final evaluation on the Test set.

B. Training Convergence

Training was conducted over 100 epochs. As shown in the loss curves, both training and validation loss converged rapidly, with MSE values stabilizing below 10^{-3} after normalization. The use of a random split for validation prevented the model from overfitting to a specific growth phase, allowing it to accurately predict the "collapse" events seen in the training data.

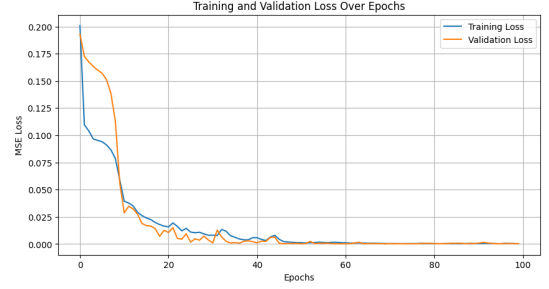


Fig. 6. Training and validation Mean Squared Error (MSE) loss over 100 epochs. The convergence of both metrics near zero suggests stable learning and effective generalization.

C. 1-Step Ahead vs. Recursive Forecasting

In 1-step ahead prediction, the model tracked the actual laser measurements with high fidelity, achieving an overlay that was nearly indistinguishable from the ground truth.

However, the recursive forecasting task (predicting 200-1,000 steps into the future) proved more challenging. In this mode, errors accumulated at each step. Models with shorter window sizes tended to converge toward a mean value (stagnation), whereas the optimized CNN-LSTM with a larger window size successfully replicated the oscillatory nature and the increasing amplitude of the signal.

D. Final Evaluation

Upon testing against the hidden test set released on May 8th, the model achieved an MSE of 815.1715 and an MAE of 20.7715. The visual comparison confirms that the CNN filters effectively pre-processed the signal, allowing the LSTM layers to maintain the periodicity of the laser pulses over the required 200-step horizon.

V. CONCLUSION

In this assignment, we developed a hybrid CNN-LSTM model to forecast non-stationary laser measurement data. Our findings indicate that while LSTMs are proficient at capturing temporal dependencies, the addition of a 1D-Convolutional layer significantly improves the model's ability to extract macro-level patterns from noisy sequences. Furthermore, we demonstrated that the choice of window size is paramount for recursive forecasting; a window that encompasses at least one full cycle of the growth-and-collapse pattern is required to prevent the model from stagnating. Future work could explore the use of Attention mechanisms to further refine the focus on critical transition points in the data.

REFERENCES

- [1] M. Abumohsen, A. Y. Owda, M. Owda, and A. Abumihsan, "Hybrid machine learning model combining of cnn-lstm-rf for time series forecasting of solar power generation," *e-Prime - Advances in Electrical Engineering, Electronics and Energy*, vol. 9, p. 100636, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S277267112400216X>